



BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

MOBİL PROGRAMLAMA DERSİ

DÖNEM PROJESİ

BOMBA İMHA OYUNU

Hazırlayanlar:

Şevval Ayça Çerence - 152120211128

Merve Selçuk - 152120211116

İçindekiler

1. Proje Özeti	3
2. Kullanılan Teknolojiler ve Araçlar	3
2.1. Geliştirme Platformu.....	3
2.2. Android SDK Bileşenleri	4
2.3. Kullanılan Kütüphaneler	4
3. Sistem Mimarisi ve Sınıf Yapısı	4
3.1. Genel Mimari Yaklaşım	4
3.2. Ana Java Sınıfları ve Görevleri	5
3.3. Veri Akışı	7
3.4. OOP Prensipleri Açısından Değerlendirme.....	7
4. Oyun Mekanikleri ve Modül Algoritmaları.....	8
4.1. Rol Ayrımı.....	8
4.2. Bomba Kontrol Paneli.....	8
4.3. Geri Sayım Sayacı	8
4.4. Başarı ve Hata Durumu.....	8
4.5. Kablo Modülü	9
4.6. Büyük Düğme Modülü	9
4.7. Renk Hafıza Modülü.....	9
4.8. Kadran Modülü	9
4.9. Şifre Paneli Modülü	9
4.10. Son 2.5 Dakika Uyarısı	10
5. UI/UX Tasarım Tercihleri.....	10
5.1. Tema ve Renk Paleti	10
5.2. Drawable Tasarımı	10
5.3. Layout Hiyerarşisi.....	11
5.4. Kullanıcı Deneyimi Açısından Güçlü Noktalar.....	11
6. Karşılaşılan Zorluklar ve Teknik Çözümler	11
6.1. Ekranlar Arası Oyun Durumunu Koruma	11
6.2. Ağ Bağımlılığı Olan Şifre Modülü.....	11
6.3. UI Thread Yönetimi	12
6.4. Görsel Stres ve Zaman Baskısı	12
6.5. Özel Sayaç Görünümü	12
6.6. Skor Geçmişi ve Veri Kalıcılığı	12
6.7. Farklı Ekran Boyutlarında İçerik Taşması.....	12
7. Sonuç ve Gelecek Geliştirmeler	13
7.1. Mevcut Durum.....	13
7.2. Gelecek Geliştirme Önerileri	13
Genel Değerlendirme	14

1. Proje Özeti

Bu proje, Bomba imha adlı Android tabanlı, asimetrik rol paylaşımına dayanan bir bomba imha oyunudur. Projede , bir oyuncu bombayı fiziksel olarak görür ve modüllerle etkileşime girer; diğer oyuncu ise rehber/uzman rolünde teknik dokümanları okuyarak doğru talimatları verir. Bu yapı, oyuncular arasında sözlü iletişim, zaman baskısı, hata toleransı ve problem çözme becerilerini merkeze alan bir oyun deneyimi üretir.

Oyunun temel amacı, geri sayım bitmeden ve izin verilen hata sayısı aşılmadan bomba üzerindeki modülleri çözmektir. Kod tabanında toplam beş güvenlik modülü tanımlıdır: kablo, büyük düğme, renk hafıza, kadran ve şifre paneli. Her modül kendi ekranında farklı kurallara göre doğru hamleyi hesaplar; yanlış hamleler `GameState.mistakeCount` değerini artırarak oyuncuyu kaybetme durumuna yaklaştırır.

Temel oyun döngüsü şu şekildedir:

1. Kullanıcı açılış ekranından rol seçimine geçer.
2. İmhacı rolü seçildiğinde oyuncu adı alınır ve yeni oyun durumu oluşturulur.
3. Bomba kontrol paneli açılır; sayaç, hata sayısı, bomba bilgileri ve modül kartları gösterilir.
4. Oyuncu modülleri tek tek açar, doğru işlemleri yapmaya çalışır.
5. Tüm modüller çözüldürse görev başarılı olur; süre biterse veya hata limiti aşılsa bomba patlar.
6. Sonuç ekranında skor, kalan süre, hata sayısı ve modül ilerlemesi raporlanır.
7. Skor geçmiş SQLLite veritabanında saklanır ve ayrı bir ekranda listelenir.

Hedef kitle; mobil oyun geliştirme, takım içi iletişim oyunları ve Android tabanlı etkileşimli eğitim/oyun projeleriyle ilgilenen kullanıcılardır. Projenin akademik açıdan güçlü yönü, yalnızca arayüz göstermesi değil, her modül için kural tabanlı algoritmalar, zaman yönetimi, kalıcı veri saklama ve özel çizim bileşenlerini bir araya getirmesidir.

2. Kullanılan Teknolojiler ve Araçlar

2.1. Geliştirme Platformu

Proje Android Studio yapısına uygun bir Gradle projesidir. `app/build.gradle` dosyasındaki yapılandırmaya göre uygulama aşağıdaki SDK ve dil özellikleriyle geliştirilmiştir:

- Programlama dili: Java
- UI tanımlama dili: XML
- Compile SDK: 36
- Target SDK: 36
- Minimum SDK: 27
- Java sürümü: Java 11
- Paket adı / namespace: `com.example.codedefuse`

2.2. Android SDK Bileşenleri

Projede kullanılan başlıca Android bileşenleri şunlardır:

- **Activity:** Her ana ekran ve modül ekranı ayrı `Activity` olarak modellenmiştir.
- **Intent:** Ekranlar arası geçişlerde kullanılmaktadır.
- **CountDownTimer:** Bomba süresini yönetmek için kullanılmaktadır.
- **Handler / Looper:** Şifre kelimesi yükleme, düğme modülü sayaç güncellemesi ve son 2.5 dk uyarı efekti gibi zamanlanmış işlemler için kullanılmıştır.
- **SQLiteOpenHelper:** Skor geçmişinin yerel veritabanında saklanması için kullanılmıştır.
- **SharedPreferences:** Kullanıcı adını kalıcı olarak saklamak için kullanılmıştır.
- **URLConnection:** Şifre modülünde güvenlik kelimesi almak için internet tabanlı API çağırısı yapılmaktadır.

2.3. Kullanılan Kütüphaneler

`app/build.gradle` içinde AndroidX ve Material bileşenleri tanımlıdır:

- `appcompat`
- `material`
- `activity`
- `constraintlayout`
- `junit`
- `espresso`

Mevcut kodun önemli bir kısmı klasik Android `Activity` ve XML layout yaklaşımıyla yazılmıştır. `MainActivity` içinde `EdgeToEdge`, `ViewCompat` ve `WindowInsetsCompat` kullanılmış olsa da uygulamanın ana başlatıcı ekranı manifestte `WelcomeActivity` olarak tanımlıdır.

3. Sistem Mimarisi ve Sınıf Yapısı

3.1. Genel Mimari Yaklaşım

Proje, Activity merkezli bir Android mimarisi kullanır. Her oyun ekranı veya rehber ekranı ayrı bir sınıfla temsil edilmiştir. Merkezi oyun durumu ise `GameState` adlı statik veri sınıfı üzerinden yönetilir. Bu yaklaşım, küçük ve orta ölçekli mobil oyun projeleri için hızlı geliştirme ve ekranlar arası veri paylaşımı açısından pratiktir.

Mimari üç ana katmanda incelenebilir:

- Sunum katmanı: XML layout dosyaları ve Activity sınıfları.
- Oyun mantığı katmanı: Modül Activity'leri ve `GameState`.
- Veri saklama katmanı: `DatabaseHelper`, `Score`, `PreferencesHelper`.

3.2. Ana Java Sınıfları ve Görevleri

Sınıf	Görev
WelcomeActivity	Açılış ekranını gösterir ve kullanıcıyı rol seçimine yönlendirir.
RoleSelectionActivity	İmhacı ve uzman/kılavuz rollerini seçtirir; skor geçmişi ve ana menü geçişlerini yönetir.
DefuserSetupActivity	Oyuncu adını alır, PreferencesHelper ile saklar ve GameState.setupGame() çağrısıyla yeni oyun başlatır.
BombActivity	Ana bomba kontrol panelidir; sayaç, hata sayısı, modül kartları, kazanma/kaybetme akışı ve patlama animasyonu bu sınıfta yönetilir.
GameState	Oyun süresi, hata sayısı, modül çözülme durumları, bomba bilgileri ve modül verilerini statik olarak tutar.
WireModuleActivity	Kablo modülündeki renk dizisini üretir ve doğru kablo algoritmasını çalıştırır.
ButtonModuleActivity	Büyük düğme modülünde düğme rengi/yazısı/indicator/pil bilgilerine göre doğru eylemi belirler.
ColorMemoryModule Activity	Renk hafıza modülünde seri numarası ve hata sayısına bağlı renk eşleştirmesi yapar.
DialModuleActivity	Kadran modülünde seri numarası, pil sayısı ve indicator bilgisine göre yön sırası oluşturur.

Sınıf	Görev
PasswordModuleActivity	Güvenlik kelimesini API veya yerel fallback üzerinden alır ve PIN doğrulaması yapar.
ExpertMenuActivity	Uzman/kılavuz rolündeki kullanıcının teknik dokümanlara eriştiği menüdür.
WireGuideActivity, ButtonGuideActivity, ColorMemoryGuideActivity, DialGuideActivity, PasswordGuideActivity	Her modül için rehber/kural ekranlarını gösterir.
ResultActivity	Oyun sonucunu, skor hesaplamasını ve veritabanına kayıt işlemini yürütür.
ScoreHistoryActivity	SQLite'tan skor kayıtlarını okur ve ListView ile gösterir.
DatabaseHelper	SQLite tablo oluşturma, güncelleme, skor ekleme, skor listeleme ve temizleme işlemlerini yapar.
Score	Skor kayıtlarını temsil eden veri modelidir.
PreferencesHelper	Kullanıcı adını SharedPreferences ile saklar ve okur.
SevenSegmentTimerView	Geri sayım sayacını özel Canvas çizimiyle yedi segmentli dijital ekran biçiminde gösterir.

Sınıf	Görev
LowTimeWarningOverlay	Son 2.5 dk ekran üzerine kırmızı yanıp sönen uyarı katmanını ekler.

3.3. Veri Akışı

Ana veri akışı aşağıdaki gibi özetlenebilir:

1. `DefuserSetupActivity`, kullanıcı adını alır.
2. `GameState.setupGame(username)` çağrılır.
3. `GameState.resetGame()` tüm modül ve bomba durumlarını sıfırlar.
4. `GameState.generateBombInfo()` seri numarası, pil sayısı ve indicator bilgisini üretir.
5. `BombActivity`, bu merkezi durum verilerini ekrana basar.
6. Modül Activity'leri `GameState` üzerinden mevcut bomba bilgilerini okur.
7. Doğru hamlede `GameState.mark...Solved()`, yanlış hamlede `GameState.addMistake()` çağrılır.
8. `BombActivity.onResume()` ile ana panel güncellenir ve oyun bitiş koşulları kontrol edilir.
9. `ResultActivity`, son durumu okuyarak sonucu ve skoru üretir.
10. `DatabaseHelper.insertScore()` ile skor kalıcı hale getirilir.

3.4. OOP Prensipleri Açısından Değerlendirme

Projede nesne yönelimli tasarımın temel örnekleri görülmektedir:

- **Sorumluluk ayrımı:** Her modülün mantığı kendi Activity sınıfında tutulmuştur.
- **Modelleme:** `Score`, veritabanı kayıtlarını temsil eden sade bir model sınıfıdır.
- **Yardımcı sınıflar:** `DatabaseHelper`, `PreferencesHelper` ve `LowTimeWarningOverlay` tekrar eden görevleri ayrıştırır.
- **Kapsülleme:** `Score` sınıfında alanlar private tutulmuş, erişim getter metotlarıyla sağlanmıştır.
- **Özel bileşen geliştirme:** `SevenSegmentTimerView`, Android `View` sınıfından türetilerek özel çizim davranışı kazanmıştır.

Bununla birlikte `GameState` tamamen statik bir durum yöneticisidir. Bu kullanım küçük oyun prototiplerinde pratiktir; ancak daha büyük projelerde lifecycle, test edilebilirlik ve süreç ölümü sonrası veri kaybı açısından `ViewModel`, repository veya Parcelable tabanlı daha dayanıklı bir mimari tercih edilebilir.

4. Oyun Mekanikleri ve Modül Algoritmaları

4.1. Rol Ayrımı

Rol ayrımı teknik olarak `RoleSelectionActivity` üzerinden yapılır. İmhacı rolü `DefuserSetupActivity` ve ardından `BombActivity` akışına gider. Uzman/kılavuz rolü ise `ExpertMenuActivity` ekranına gider ve buradan modül rehberlerine erişilir.

Bu ayrım asimetrik oyun yapısını destekler. İmhacı gerçek bomba panelini ve modül etkileşimlerini görürken, uzman tarafı yalnızca teknik dokümanlara erişir. Böylece iki rol arasında doğrudan bilgi asimetrisi oluşturulur.

4.2. Bomba Kontrol Paneli

`BombActivity`, oyunun merkezidir. Bu ekranda:

- kalan süre,
- hata sayısı,
- çözülen modül sayısı,
- seri numarası,
- pil sayısı,
- indicator bilgisi,
- beş aktif modül kartı gösterilir.

Her kart kendi modül Activity'sine geçiş yapar. Eğer modül zaten çözülmüşse kullanıcı Toast mesajıyla bilgilendirilir.

4.3. Geri Sayım Sayacı

Ana sayaç `BombActivity` içinde `CountDownTimer` ile yönetilir. Her saniyede `GameState.remainingTime` güncellenir, ekran yenilenir ve oyun bitiş koşulları kontrol edilir. Sayaç görünümü standart `TextView` yerine `SevenSegmentTimerView` ile çizilir. Bu özel görünüm, yedi segmentli dijital saat estetiği vererek bomba temasını güçlendirir.

4.4. Başarı ve Hata Durumu

Yanlış hamleler `GameState.addMistake()` ile merkezi hata sayısını artırır. Hata sayısı maksimum değere ulaştığında oyun kaybedilmiş kabul edilir.

Başarı durumu ise her modülün ilgili `mark...Solved()` metodu ile çözülmüş olarak işaretlenmesi ve toplam çözülen modül sayısının `totalModuleCount` değerine ulaşmasıyla oluşur.

4.5. Kablo Modülü

Kablo modülü, rastgele 3 ila 6 arasında kablo üretir ve her kabloya renk atar. Doğru kablo, kablo sayısı, renk sayıları ve seri numarasının son rakamının tek/çift olması gibi kurallarla belirlenir.

Bu modül, klasik kural tabanlı karar ağacı yaklaşımıyla uygulanmıştır.

4.6. Büyük Düğme Modülü

Buton modülü; düğme rengi, düğme yazısı, pil sayısı ve indicator değerine göre iki temel eylemden birini seçtirir:

- hemen bas,
- basılı tut.

Basılı tutma senaryosunda şerit rengi üretilir ve kullanıcının doğru sayaç rakamında bırakması beklenir. Sayaç `Handler` ve `Runnable` ile periyodik güncellenir.

Bu tasarım, tek modül içinde iki aşamalı karar mekanizması kurar.

4.7. Renk Hafıza Modülü

Renk hafıza modülü üç aşamalı bir sıra üretir. Kullanıcının basması gereken doğru renk, yanan renk, mevcut hata sayısı ve seri numarasında sesli harf bulunup bulunmamasına göre hesaplanır.

Bu modül, oyun durumuna bağlı dinamik kural tablosu kullanır.

4.8. Kadran Modülü

Kadran modülü üç adımlı yön dizisi üretir. Dizinin oluşturulmasında indicator, pil sayısı, seri numarasının son rakamı ve seri numarasında sesli harf bulunması kullanılır.

Bu yapı, sıralı giriş doğrulama mantığını gösterir.

4.9. Şifre Paneli Modülü

Şifre modülü, güvenlik kelimesi üzerinden PIN hesaplatır. Güvenlik kelimesi öncelikle internet üzerinden alınır; API çağrısı başarısız olursa yerel fallback kelime listesi kullanılır. Bu yaklaşım, ağ bağımlılığına karşı dayanıklılık sağlar.

PIN hesabında:

- seri numarasının ilk rakamı,
- pil sayısı,
- indicator,
- kelime uzunluğu,
- kelimenin sesli harfle başlayıp başlamaması,
- kelimedeki `R` harfi bulunması

kullanılır.

4.10. Son 2.5 Dakika Uyarısı

Projede son 2.5 dk için 'LowTimeWarningOverlay' adlı ortak bir uyarı bileşeni oluşturulmuştur. Bu sınıf ilgili Activity'nin root içeriğine kırmızı bir 'View' ekler ve 'ObjectAnimator' ile alpha değerini değiştirerek yanıp sönme efekti üretir.

Bu çözüm, modül ekranlarında da son saniye stresini sürdürür ve oyuncunun zaman baskısını hissetmesini sağlar.

5. UI/UX Tasarım Tercihleri

5.1. Tema ve Renk Paleti

Projenin görsel kimliği 'colors.xml' dosyasında belirgin biçimde tanımlanmıştır. Arayüzde koyu arka plan, neon camgöbeği, neon yeşil ve kırmızı alarm tonları kullanılmaktadır.

Öne çıkan renkler:

- 'bg_dark': koyu teknoloji arka planı.
- 'panel_dark': panel/kart zeminleri.
- 'neon_green': başarı, aktif sistem ve pozitif durumlar.
- 'neon_red': hata, patlama ve risk durumları.
- 'neon_yellow': kodda camgöbeği tonuna karşılık gelen vurgu rengi.
- 'text_main' ve 'text_soft': okunabilirlik için ana ve ikincil metin renkleri.

Bu palet, siberpunk/dark tech temasını teknik paneller, neon kenarlıklar ve dijital sayaçlarla destekler.

5.2. Drawable Tasarımı

Drawable XML dosyaları, uygulamanın görsel dilini oluşturur. Örneğin:

- 'bg_panel.xml': karanlık panel zemini ve yumuşak kenarlık.
- 'module_solved.xml': çözülen modüller için yeşil temalı kart.
- 'module_unsolved.xml': aktif/çözülmemiş modüller için koyu panel ve neon çerçeve.
- 'btn_neon_green.xml', 'btn_neon_red.xml', 'btn_neon_yellow.xml': durum odaklı butonlar.
- 'bg_simon_module_casing.xml', 'bg_simon_panel_rim.xml': renk hafıza modülü için donanım hissi veren çerçeveler.
- 'bg_dial_center.xml', 'bg_dial_slot.xml': kadran modülünün fiziksel arayüzünü destekleyen özel şekiller.

Bu yapı, bitmap görseller ve XML shape dosyalarını birlikte kullanarak tematik bütünlük sağlar.

5.3. Layout Hiyerarşisi

Projede çoğunlukla `ScrollView`, `LinearLayout`, `FrameLayout` ve bazı ekranlarda `RelativeLayout` kullanılmıştır. Bu tercih, mobil ekranlarda dikey akışlı oyun panelleri için uygundur.

Öne çıkan layout tercihleri:

1. **ScrollView:** Uzun modül ve sonuç ekranlarında içerik taşmasını yönetmek için kullanılır.
2. **LinearLayout:** Kartların ve form elemanlarının dikey/horizontal sıralanmasını sağlar.
3. **FrameLayout:** Patlama görseli, kırmızı uyarı overlay'i ve modül donanım panelleri gibi üst üste binen arayüz öğelerinde kullanılır.
4. **ConstraintLayout:** `activity\_main.xml` içinde yer alır; ancak gerçek başlatıcı akışında aktif rolü sınırlıdır.
5. **RelativeLayout:** Açılış veya rol seçimi gibi arka plan görseli + içerik katmanı ihtiyacı olan ekranlarda kullanılır.

5.4. Kullanıcı Deneyimi Açısından Güçlü Noktalar

- Karanlık teknoloji teması oyun konusu ile uyumludur.
- Modül kartları çözüldüğünde görsel olarak değişir.
- Son 2.5 dakikada kırmızı yanıp sönmeye efekti zaman baskısını güçlendirir.
- Patlama durumunda sonuç ekranından önce görsel patlama bildirimi gösterilir.
- Rehber ekranları uzman/kılavuz rolü için ayrı bir bilgi katmanı sağlar.
- Skor geçmişi oyuncuya performans takibi sunar.

6. Karşılaşılan Zorluklar ve Teknik Çözümler

6.1. Ekranlar Arası Oyun Durumunu Koruma

Birden fazla Activity kullanıldığında oyun verilerinin ekranlar arasında taşınması zorlaşır. Projede bu sorun `GameState` adlı merkezi statik sınıfla çözülmüştür. Modül ekranları aynı global durumu okuyup güncelleyebilir.

Uygulanan çözüm: Statik alanlar üzerinden merkezi oyun durumu yönetimi.

Değerlendirme: Hızlı ve anlaşılırdır; ancak uygulama süreçten düşerse durum kaybı yaşanabilir. Daha ileri sürümde `ViewModel`, `SavedStateHandle` veya kalıcı oyun oturumu kullanılabilir.

6.2. Ağ Bağımlılığı Olan Şifre Modülü

Şifre modülü güvenlik kelimesini internetten almaya çalışır. Mobil oyunlarda ağ gecikmesi veya bağlantı yokluğu kullanıcı deneyimini bozabilir.

Uygulanan çözüm: Birincil API başarısız olursa ikinci API denenir; o da başarısız olursa yerel kelime listesi kullanılır.

```
```java
private static final String[] FALLBACK_WORDS =
 {"ORBIT", "PANEL", "SIGNAL", "VECTOR", "MODULE", "CIRCUIT", "ACCESS"};
```
```

Bu yaklaşım modülün çevrimdışı durumda da oynanabilmesini sağlar.

6.3. UI Thread Yönetimi

Android'de ağ istekleri ve zamanlayıcılar doğrudan UI thread üzerinde çalıştırılırsa donma veya hata oluşabilir.

Uygulanan çözüm:

- API çağrısı `new Thread(...)` içinde yapılır.
- UI güncellemesi `runOnUiThread(...)` ile ana thread'e alınır.
- Periyodik sayaçlar `Handler` ve `Looper.getMainLooper()` ile yönetilir.

Bu kullanım, temel Android thread güvenliği açısından doğrudur.

6.4. Görsel Stres ve Zaman Baskısı

Oyunun temasında süre baskısı önemlidir. Sadece sayacın azalması oyuncu gerilimini yeterince hissettirmeyebilir.

Uygulanan çözüm: `LowTimeWarningOverlay` ile son 2.5 dakikada tüm ilgili ekranlarda kırmızı yanıp sönme efekti oluşturulmuştur. Ortak yardımcı sınıf kullanılması, kod tekrarını azaltır.

6.5. Özel Sayaç Görünümü

Standart metin tabanlı sayaç, bomba imha teması için yeterince etkileyici olmayabilir.

Uygulanan çözüm: `SevenSegmentTimerView`, Canvas ile dijital yedi segmentli sayaç çizer. Bu, arayüzün oyun atmosferini güçlendirir.

6.6. Skor Geçmişi ve Veri Kalıcılığı

Oyuncu performansının korunması için kalıcı veri saklama gerekir.

Uygulanan çözüm: `DatabaseHelper`, SQLite tablosu oluşturur ve skor kayıtlarını yerel veritabanında saklar. `ScoreHistoryActivity` ise bu kayıtları listeler.

6.7. Farklı Ekran Boyutlarında İçerik Taşması

Bomba kontrol paneli ve sonuç ekranı gibi uzun içerikli sayfalarda küçük ekranlarda alt bölümün sistem navigasyon çubuğunun altında kalması olasıdır.

Uygulanan çözüm: `ScrollView`, `fitsSystemWindows`, ek alt padding ve görünür scrollbar kullanımlarıyla içerik erişilebilirliği artırılmıştır.

7. Sonuç ve Gelecek Geliştirmeler

7.1. Mevcut Durum

Bomba imha oyunu, Android üzerinde çalışan, çok ekranlı ve kural tabanlı bir bomba imha oyunudur. Projede beş farklı modül, iki rol akışı, skor sistemi, yerel veri saklama, özel sayaç çizimi ve tematik arayüz bileşenleri bulunmaktadır. Oyun, akademik bir mobil programlama projesi için iş mantığı, UI tasarımı, veri yönetimi ve kullanıcı etkileşimi açısından yeterli çeşitliliğe sahiptir.

Güçlü yönler:

- Net rol ayrımı.
- Birden fazla algoritmik modül.
- Merkezi oyun durumu yönetimi.
- SQLite skor geçmişi.
- Siberpunk/dark tech temaya uygun görsel tasarım.
- Zaman baskısını artıran alarm ve patlama geri bildirimi.

7.2. Gelecek Geliştirme Önerileri

1. Gerçek zamanlı multiplayer bağlantısı: İmhacı ve uzman/kılavuz farklı cihazlardan oynayabilir. Bunun için WebSocket, Firebase Realtime Database veya Bluetooth/Wi-Fi Direct seçenekleri değerlendirilebilir.
2. Oyun durumunun daha dayanıklı saklanması: `GameState` yerine `ViewModel`, repository ve `SavedStateHandle` tabanlı bir yapı kurulabilir.
3. Yeni modüller: Morse kodu, sembol eşleştirme, labirent, frekans ayarı veya seri devre modülleri eklenebilir.
4. Zorluk seviyeleri: Süre, hata hakkı, modül sayısı ve modül karmaşıklığı seviyeye göre değiştirilebilir.
5. Gelişmiş skor tablosu: En yüksek skorlar, filtreleme, kullanıcı bazlı kayıt ve başarı rozetleri eklenebilir.
6. Ses ve titreşim geri bildirimi: Son 2.5 dk , hata yapma ve patlama anı için ses/titreşim desteği oyunun gerilimini artırabilir.
7. Erişilebilirlik iyileştirmeleri: Renk körlüğü modları, yüksek kontrast seçenekleri ve metin boyutu ayarları eklenebilir.
8. Test altyapısı: Modül algoritmaları için birim testler yazılarak doğru kablo, doğru renk, doğru yön ve PIN hesaplama kuralları otomatik doğrulanabilir.
9. Kod mimarisi iyileştirmesi: Modül algoritmaları Activity dışına taşınarak ayrı servis/sınıf yapısında test edilebilir hale getirilebilir.

Genel Değerlendirme

Bomba imha&uzman, mobil oyun geliştirme açısından hem etkileşimli UI tasarımını hem de kural tabanlı oyun mantığını barındıran kapsamlı bir Android projesidir. Kod tabanı, Activity tabanlı klasik Android geliştirme yaklaşımını izlemekte; oyun kuralları, kullanıcı etkileşimi, veri kalıcılığı ve tematik görsel tasarım bileşenlerini birlikte kullanmaktadır. Proje, akademik sunum ve değerlendirme açısından oyun mekanikleri, sınıf sorumlulukları, Android lifecycle kullanımı ve yerel veri yönetimi başlıklarında güçlü bir inceleme zemini sunar.